HOOGHLY ENGINEERING & TECHNOLOGY COLLEGE

Final Year Project
(2022-2026)

Web Development

Decentralized Machine Learning Training Platform (PaaS)

TEAM MEMBERS

Name	University Roll no
Akash Poddar	17600122071
Debjit Mitra	17600122046
Soham De	17600122062
Sourodip Ghosh	17600122040

Mentored by ***Assistant Professor Agnimita Banerjee***

PROBLEM STATEMENT

The project addresses the fundamental challenge of inefficient and costly resource utilization in Machine Learning (ML) model training, specifically for small-to-medium enterprises (SMEs), researchers, and independent developers.

Core Problem

- Cost & Inflexibility: Centralized cloud platforms charge premium, fixed rates, leading to wasted expenditure when resources are not fully utilized.
- Untapped Potential: A vast, globally distributed pool of powerful, idle compute resources (GPUs) remains underutilized.

FACTS AND FIGURES

- Cloud GPUs cost 2–10 USD per hour, making even mid-sized model training reach ₹15,000–₹50,000 per run. GPU hardware prices have risen 200% (2020–2024), putting local GPU ownership out of reach for most learners and startups.
- AI compute needs are growing 2× every six months, far faster than hardware availability. 70% of ML students report they cannot train models effectively due to hardware limits, and 45% of companies face compute-related project delays.
- Only 10–15% of ML learners have dedicated GPUs, forcing 80%+ to rely on slow laptops. Training on low-end devices can take 10–20× longer, often extending model training from hours to several days.
- Companies locked into a single cloud provider spend 30–40% more over time. High pay-per-use GPU pricing makes cloud-based training unaffordable for students and independent developers.

OUR SOLUTION

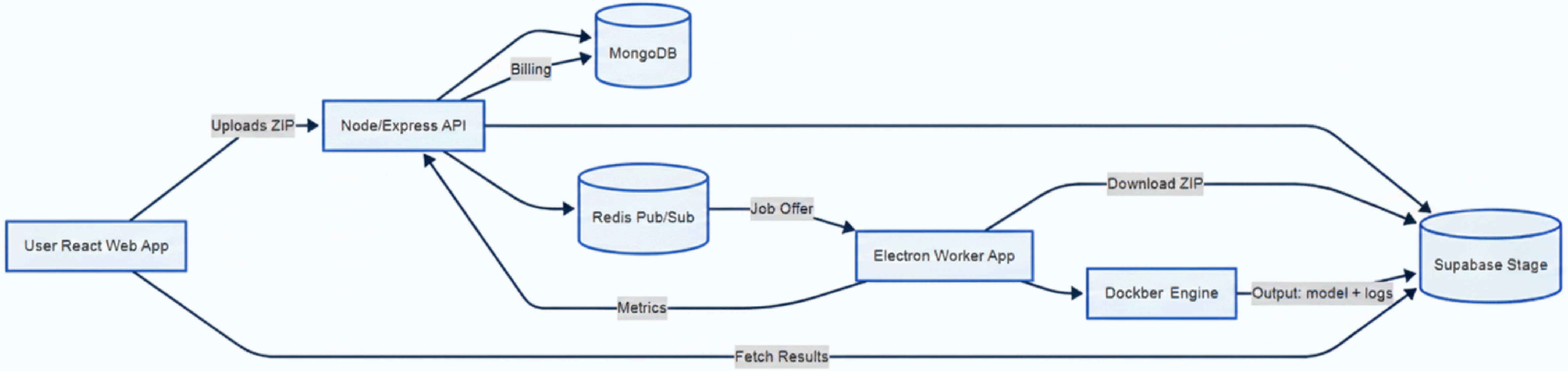
To democratize Machine Learning (ML) model training and significantly reduce its cost by:

- Creating a secure, decentralized, consumption-based marketplace (pay-as-you-go).
- Efficiently connecting ML model trainers (Job Creators) with a global network of underutilized computational resources (Resource Providers/Worker Nodes).

How do we plan to achieve it?

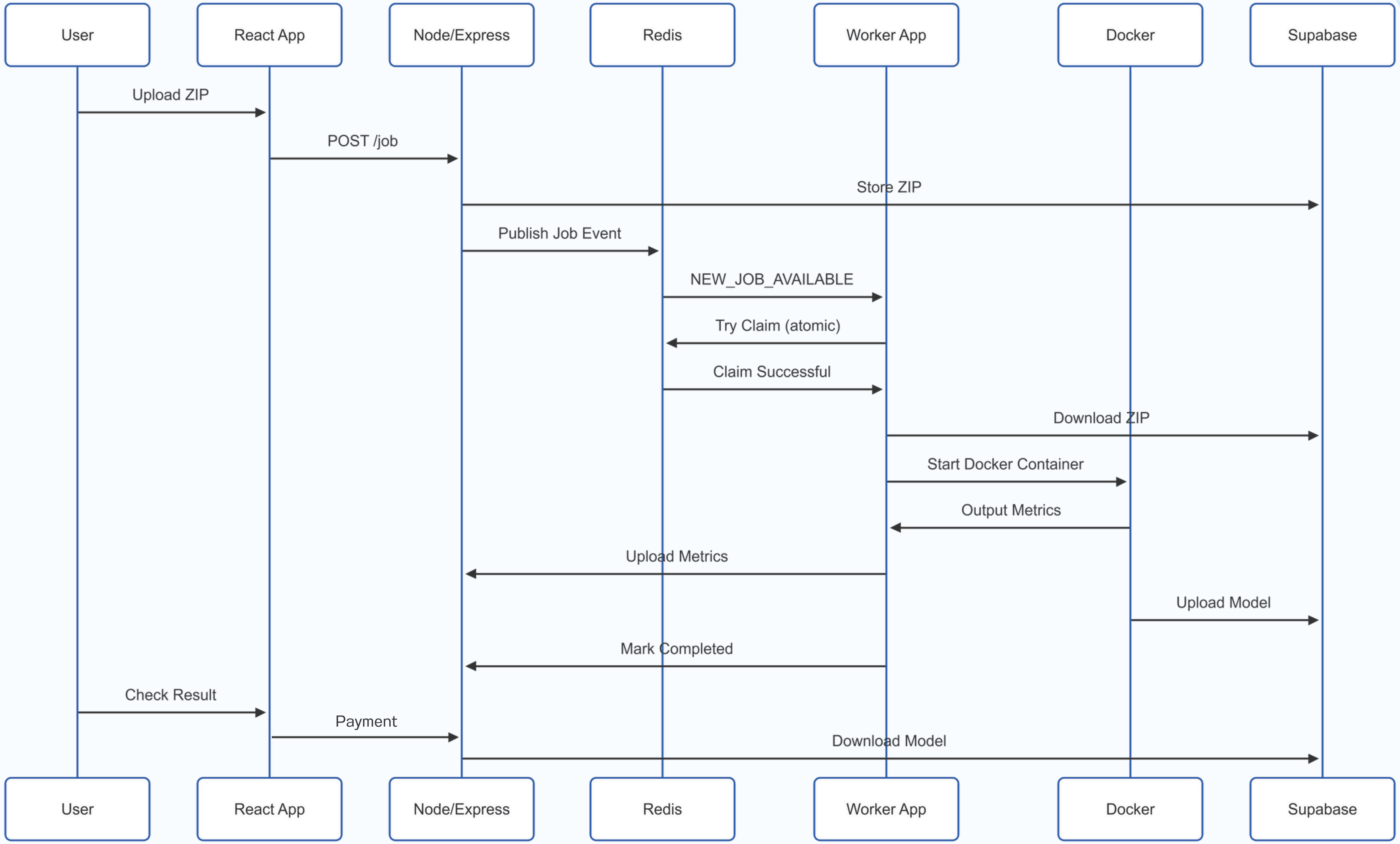
- By creating a platform where users upload a ZIP file containing their training code, data, and dependencies.
- A free worker node on the network picks up the job and trains the model on the user's behalf.
- Once training is complete, the user can download the generated model.
- They only pay for the resources consumed by the worker without the need for a high-end PC of their own.

SYSTEM ARCHITECTURE



- The user uploads a ZIP file (dataset or training job) from the React web app, which is then sent to the Node/Express API.
- The API stores metadata in MongoDB, saves the ZIP file to Supabase Storage, and publishes a job request through Redis Pub/Sub.
- The Electron Worker App, subscribed to Redis, receives the job request notification. If any worker present in the network accepts the job, then the ZIP and the metadata are extracted.
- The Worker uses Docker to run an isolated ML training container using the downloaded data.
- During training, Docker outputs logs, which are streamed to the frontend. After training completion, the Worker uploads the model back to Supabase Storage.
- The Worker sends training metrics and usage info to the API, which updates billing and job status in MongoDB.
- The user can then fetch results (model files, logs, metrics) directly from Supabase via the React app.

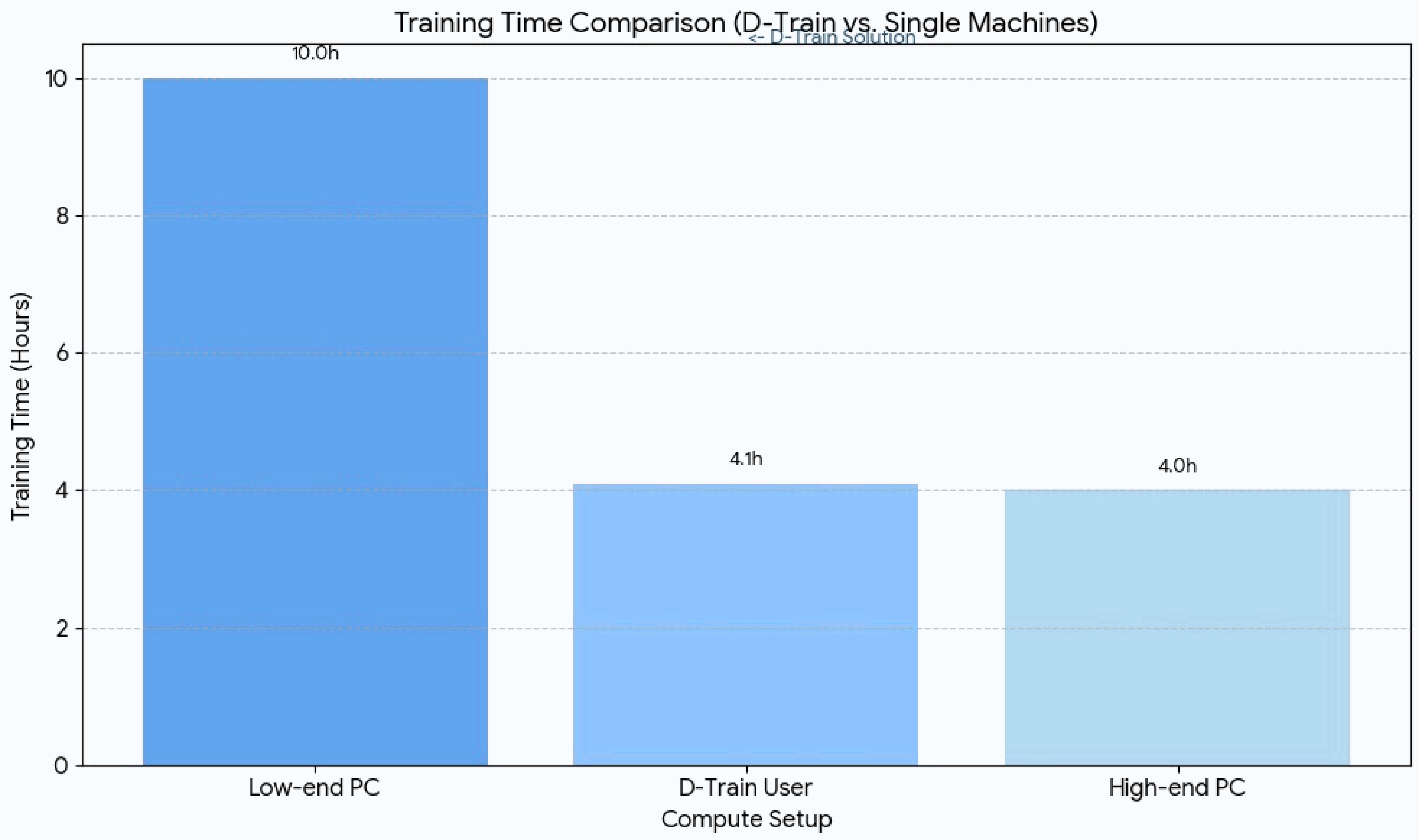
WORKFLOW (SEQUENCE)



TECH STACK

Component	Technology Used	Purpose
API & Job Orchestration	Node.js, Express.js	*Excellent for high concurrency*
Database (Primary)	Mongo DB	*Handle unstructured/semi-structured job-related data efficiently.*
Real-Time Job Queue	Redis	*Fast job dispatch, instant worker communication, reliable availability tracking, and async coordination.*
File Storage	Supabase Object Storage	*Secure and scalable object storage for large training files.*
Platform Frontend	React js	*For Clean and Responsive UI.*
Worker Software	Electron + Node js	*Platform-independent app to integrate current technology in Mac, Win and Linux.*
Training Container	Docker	*to run every training job in a clean, isolated, reproducible environment*

STATISTICS & CONCLUSION



D-Train tackles the lack of accessible, affordable ML compute by using distributed worker nodes, Docker isolation, and real-time Redis orchestration to form a lightweight community-powered training network. Users submit a job, workers do the heavy lifting, and everyone pays only for what they use, without needing a high-end PC.

THANK YOU